



浙江工业大学

本科毕业设计（论文）

设计(论文)

题 目 _____ 基于心电散点图识别的心脏类疾病诊断 _____

指导教师 _____ 谢聪聪 _____

学 生 _____ 杨德乾 _____

学 号 _____ 201806110319 _____

院 系 _____ 理学院 _____

专 业 _____ 信息与计算科学 _____

班 级 _____ 1801 _____

浙江工业大学 本科生毕业设计(论文、创作)诚信承诺书

本人慎重承诺和声明.

1. 本人在毕业设计（论文、创作）撰写过程中，严格遵守学校有关规定，恪守学术规范，所呈交的毕业设计（论文、创作）是在指导教师指导下独立完成的。
2. 毕业设计（论文、创作）中无抄袭、剽窃或不正当引用他人学术观点、思想和学术成果，无虚构、篡改试验结果、统计资料、伪造数据和运算程序等情况。
3. 若有违反学术纪律的行为，本人愿意承担一切责任，并接受学校按有关规定给予的处理。

学生（签名）.

年 月 日

浙江工业大学

本科生毕业设计（论文、创作）任务书

专业. 信计 班 级. 1801 学生姓名/学号. 杨德乾/201806110319

一. 设计（论文、创作）题目. 基于心电散点图识别的心脏类疾病诊断

二. 主要任务与目标. 主要任务是利用心电散点图技术并结合深度学习对于不同心血管疾病进行分类识别，并使用不同模型来提高识别的准确度。

三. 主要内容与基本要求. 心电信号（ECG）是一种非平稳的生理信号，代表心脏的电活动。它不仅可用于查找心跳中的病理模式，还可以用于测量心跳的规律性以及诸如精神压力之类的其他状况。心电散点图通过将动态心电图中的 RR 间期化为图像坐标，在坐标系中迭代作图制成的心电图图像。通过绘制心电散点图能够将充分利用心电数据，将一般心电图难以识别的疾病心电信号变得更加的突出。要求熟练运用 pytorch 框架搭建常见的神经网络模型，并在计算机上进行实现。

四. 计划进度.

2021 年 12 月 9 日至 2022 年 2 月 26 日 完成开题报告、文献综述和英文文献翻译

2022 年 3 月 1 日至 2022 年 3 月 20 日 完成论文整体设计和程序框架

2022 年 3 月 22 日至 2022 年 5 月 8 日 进行具体代码的书写与实验，并且完成论文初稿

2022 年 5 月 10 日至 2022 年 6 月 7 日 论文完稿

五. 主要参考文献.

1、Trieu Hai-Nguyen Le.[C]Feature Extraction Techniques for Automatic Detection of Some Specific Cardiovascular Diseases Using ECG:A Review and Evaluation Study(2020) 543-549

2、[1]霍铖宇.[D]基于庞加莱散点图和递归图的心率变异性研究(2014) 1-130

3、R.Banerjee, A.Ghose ,S.Khandelwal, [C] A Novel Recurrent Neural Network Architecture for Classification of Atrial Fibrillation Using Single-lead ECG, European Signal Processing Conference. (2019) 1-5.

任务书下发日期

2021 年 12 月 9 日

设计（论文、创作）工作自

2021 年 12 月 9 日至 2022 年 6 月 7 日

设计（论文、创作）指导教师

学科（方向）负责人

主管院长

基于心电散点图识别的心脏类疾病诊断

学生姓名. 杨德乾 指导教师. 谢聪聪、张立军教授

浙江工业大学理学院

摘 要

本论文主要研究室上性心律失常、室速性心律失常、房颤三种疾病之间的诊断分类。对于心电信号而言，利用最近几十年发展出来的混沌领域可以更好的对其进行解释以及分析。本文在简要介绍混沌相关理论以及通过实验数据证明了人类的心电数据确实为混沌序列。另外提出了一种在心电散点图领域的三维图像增强方法，利用神经网络对二维以及三维图像做了对比试验，三维图像识别率高于二维图像，可以看出三维图像相对于二维图像确有一定的优势。

关键词. Lorenz plot poicare plot RR interval 心电散点图 图像增强

Heart disease diagnosis based on RRplot identification

Student: 杨德乾

Advisor: Dr. 谢聪聪

College of Science
Zhejiang University of Technology

Abstract

This paper mainly studies the diagnostic classification of supraventricular arrhythmia, ventricular tachyarrhythmia and atrial fibrillation. For ECG signals, the chaotic field developed in recent decades can be better explained and analyzed. This paper briefly introduces the theory of chaos and proves that the human ECG data is indeed a chaotic sequence through experimental data. In addition, a three-dimensional image enhancement method in the field of ECG scattergram is proposed. The neural network is used to compare the two-dimensional and three-dimensional images. The recognition rate of the three-dimensional image is higher than that of the two-dimensional image. It can be seen that the three-dimensional image is better than the two-dimensional image. Third dimensional Images do have certain advantages.

Keywords. Lorenz plot poincare plot RR interval ECG Scatter Plot Image Augmentation

目录

中文摘要	5
英文摘要	6
第一章 绪论	10
1.1、研究动机与目的	10
1.2、研究背景	10
1.3、研究方法	10
1.4、研究内容与概述	11
第二章 混沌理论	12
2.1 混沌理论提出的背景	12
2.2 初值的敏感性	13
2.3 吸引子	13
2.4 分形	14
2.5 心电信号	15
2.5.1 相空间重建	16
2.5.2 寻找最近邻	16
2.5.3 心电数据应用	17
第三章 心电散点图	19
3.1、何为心电散点图	19
3.2、本文提出的改进方法	20
步骤 1.心电波形图数据预处理	21
步骤 2.建立二维心电散点图	21
步骤 3.统计相同坐标散点的频次	22
步骤 4.建立三维心电散点图	22

步骤 5.三维散点图切分	23
步骤 6.二维散点图图像滤波.....	23
步骤 7.三维散点图重建	23
3.3、训练神经网络验证三维心电散点图优势	23
第四章 总结与展望	25
参考文献	26
致 谢	28
附 录	29

表 1.三种疾病选取出的 m 与 j 对应表格	16
表 2.室上性、室速性心力衰竭、阵发性、长期房颤混沌性质验证的斜率表格	17
表 3.三种疾病绘制而成的二维或三维图像数量	23
表 4.三种疾病在相同分类器 VGG 下在测试集的准确率	23
图 1.Lorenz 奇异吸引子	12
图 2.豪斯多夫维度图解	13
图 3.室上性、室速性心力衰竭、阵发性、长期房颤混沌性质验证结果图	19
图 4.ECG 信号对应的波形名称	19
图 5.1 RR 间期	19
图 6. 三维散点图像作图方法	20
图 7.二维心电散点图	21
图 8.三维心电散点图	21

第一章 绪论

1.1、研究动机与目的.

人活在世上，总会对生存与死亡的话题感兴趣，而造成死亡的很大一个因素就是心脏病，于是研究其中的奥秘会是我很大的一个动机。在现在这个信息化与智能化的时代，心电数据都会进行数字化，于此同时拥有一些数学与计算机的知识，对心脏病的研究与刻画有着很大的帮助。本文的目的在于提出一种新型的三维心电散点图像处理方法，以期辅助医生诊疗。

1.2、研究背景.

人类心脏活动可视为一种混沌现象，很多实验表明，心脏的生理节律既不同于周期震荡，也非完全随机，而是具有混沌特性，在确定系统中表现为貌似随机的行为。RR 间期是指人类心跳两个 R 波之间的时间差。其数值可以表示人类心跳的一次心动周期时间。心电散点图通过将动态心电图中的 RR 间期化为图像坐标，在坐标系中迭代作图制成的心电图图像。通过绘制心电散点图能够将充分利用心电数据，将一般心电图难以识别的疾病心电信号变得更加的突出。

在混沌理论与心电学的结合过程中，初始阶段就有人试图建立疾病的数学模型，2000 年至今，国外研究未见显著进展，而心电散点图技术多由国内学者所领跑。李方洁所著的《心电散点图》在心电图领域首次引进了国外心电散点图的分析方法，并致力于制作分析软件进行大数据分析。提出了利用海量数据分析的概念。对于每一张图像数据大量重叠的问题没有过多的关注。

在 2011 年之前，国内外心电图软件研发已有一定进展，但由于人们对心电散点图作用不够了解，其软件功能也未得到完全的开发。2011 年，中国心电学会心电散点图工作委员会成立，心电散点图分析软件也逐渐受到关注。渐渐实现了实时波形回放，计算，批处理等功能。但如何进一步就已绘制形成的心电散点图进行自动化诊断，有待于进一步的研究。

在 2018 年，李方洁医生在《心电散点图—大数据分析的新视野》一文中提出了使用人工智能算法识别心电散点图图像，并对其应用前景做出了美好的展望。在现代计算机技术的支撑下，使得动态心电分析图像更加得容易，也更加方便。但由于坐标范围较小而海量数据对数据量的要求较大，二维心电散点图会对数据信息造成极大的浪费。所以如何充分利用海量数据，对心脏病实现更高精度的识别是需要进一步探索的。

1.3、研究方法.

对于收集到的数据首先对于 RR 间期的混沌性质需要验证，对于收集到的 RR 间期数据需要确认其为混沌数据，在此基础上才可以有效的对数据做进一步的处理。其中计算混沌序列的最大 Lyapunov 指数是一个简单有效的办法，如果可以计算出其为正数，即可推断混沌序列的混沌性质。然后将验证过的数据绘制成为二维以及三维心电散点图。利用计算机视觉方法进行机器学习，对比获得两种散点图的准确率。

1.4、研究内容与概述.

研究内容为通过收集 MIT-BIH 数据库中的心电数据, 如室上性心律失常、室速性心律失常、房颤三种数据集, 通过下载三种数据包, 利用 MIT-BIH 数据库中提供的 wfdb 数据处理包对原始数据进行处理。通过 qrs 寻波算法, 做一阶差分之后可以获得每种数据中各自的 RR 间期, 至此将数据处理完成。之后对其验证混沌性质, 主要使用方法为计算最大 Lyapunov 指数, 随后将获取的 RR 间期混沌时间序列数据绘制二维以及三维散点图, 其中绘制的三维散点图是作者基于二维散点图提出的一种新型的散点图绘制方法。最后利用神经网络分类器进行分类比较, 在收集的三种数据集中进行训练, 可以得出三维心电散点图相较于二维心电散点图的识别率更高。

本文章节分布为:

第一章绪论.主要呈现研究背景研究方法内容以及正文的研究脉络。

第二章混沌理论.主要介绍基础的混沌理论是什么, 从混动力系统对于初始值的敏感性开始(2.1), 介绍了从几何角度刻画混沌系统的分形(2.3), 这是一种与现实世界拟合很好的一种几何图形, 按照扩展的维度定义, 我们可以知道一个混沌系统中产生的几何图形吸引子(2.2)一定是分形, 且具有分形的诸多性质。而吸引子是一个动力系统最后的归宿, 常规的牛顿力学中的三类吸引子被称作平凡吸引子, 从混沌系统中脱胎的吸引子被称作是奇异吸引子, 奇异的点在于它是一个分形结构。从这个分形结构中我们可以较为直观的对于混沌系统进行观察。在此章节的最后我对后续进行研究的 RR 间期数据做了一个验证实验(2.4), 主要是验证心电数据中 RR 间期数据的混沌性质, 验证方法为文献[9]中所呈现算法。通过计算最大 Lyapunov 指数并将其 验证在下文对于 RR 间期进行绘图时可以保证数据的处理是正确的。

第三章主要介绍心电散点图, 首先是 3.1 介绍何为心电散点图, 简介了对于心电信号我们现有的一些数据特征提取的手段, 而心电散点图是其中较为便利且效果较好的一种手段。3.2 节针对李方洁[3]提出的二维心电散点图进行改进, 提出了三维心电散点图。对于病人长时间的心电数据, RR 间期数据值都在一个有限的范围中, 在绘图过程中有很多散点都处于相空间图的同一个位置, 而这些点的数量在二维心电散点图中被忽略了, 我们的创新点在于将这个数量利用起来, 绘制了类似等高线图的散点图。3.3 节利用目前成熟的神经网络分类器 VGG16 模型对于收集的心电数据进行分类对比实验, 对于二维散点图以及三维散点图分别进行训练得到每种疾病最后识别正确的图片在同类别图片中所占比例, 部分验证了提出方法的有效性。

第二章 混沌理论

随着数理科学发展迅速，目前为止可用于描述自然界的模式大致有三种，分别是确定论，概率论，混沌论。

从 1687 年 Isaac Newton 出版了《自然哲学的数学原理》之后，人类对于天体物理的了解登上了一个很大的台阶，牛顿利用微积分，万有引力定律以及牛顿三大定律可以精确的对天体的轨道进行数值上的刻画，乃至预测未来的运行情况，宣告了物质的运动受到某些普遍的运动规律的影响。法国数学家 Verrier 于 1846 年谁通过研究天王星的轨道以及牛顿、开普勒定律的区别时，通过计算数学的方法计算出了海王星的可能坐标，同时将自己计算的坐标发给另一位数学家 Galle 要求他观测验证，最后精准的预测出了海王星的存在且与实际轨道坐标相差 1° 左右。从此太阳系的行星从 7 颗更新为了 8 颗。这是计算数学的胜利，也是牛顿力学的胜利，至此，人们会偏向于相信大部分的运动都可以由某个确定的公式定理所主导，同时也为确定论找到一个非常有利的论据。此类二体问题可以由牛顿的万有引力定律加上微积分进行求解，但这时三体问题(太阳，地球、月球的相互影响运行轨迹问题)的研究并没有一个唯一的答案。

人们或许会思考，在天上的海王星都可以被精确的预测到，那一个人的人生轨迹是否也会被预测，这些东西是不是可以通过某个公式被准确的计算出来，一个人的未来轨迹如果从出生那时的动量、质量等物理量就可以被准确的预测出来，那人类的自由意识，努力等所谓的后天因素的意义是什么。这些都是确定论所带来的哲学上的疑问。与之十分不同的另一种思想用以拟合自然界的理论，是以概率论、随机过程等科学的发展带来的随机论的理论解释。随机论是对自然逼近的另一套完全严格的方式，理论相信物质的运动背后并不一定是一个确定的公式定理不断的作用导致物质的运动，而是由分布确定的，不同分布会有不同的概率导致物质的运动，最后导致了物质运动的随机性。到目前为止对于这个理论方向最火热的研究大概是关于统计学习、机器学习方向，对于数据的产生的随机性建立了一套严格的随机论。

但是自然界不是完全确定论的，也不是完全概率论的，确定论和概率论都是某种极限，在客观世界中并不完全成立的。而更好描述自然界的一种理论叫做混沌(chaos)理论。讨论在这个自然界中，如何在看似无规律的信息中发现一些有规律的东西。

2.1 混沌理论提出的背景

1887 年，法国数学家 Henry Poincaré 在瑞典国王的生日宴会上提交了一份关于三体问题的论文[1]发表于《Acta Mathematica》，提出了三体问题是不可解的，因为任何一点初始值的微小变动都会导致最终的结果完全不一样，无法对轨迹进行预测。Poincaré 运用自己发明的相图理论解释了其中的原理并为混沌理论的开辟奠定了基础。1948 年，E.N.Lorenz 教授在利用老式计算机计算描述天气系统的常微分方程数值解时，等待一段时间后发现机器宕机了，只好将计算完成一部分的纸带当作初始值再次输入微分方程系统中进行计算，最后的结果却是完全不一致，这时他意识到宕机时计算机内部所存储的数与打印出来的数之间相差了非常小的一个数，导致了最后的结果完全不一致。这一现象后来被称作蝴蝶效应，或是混沌系统对初始值的敏感性。

现代意义上的混沌的定义是由华裔美国数学家李天岩和其导师 Yorke 首先提出的。1975 年 Li-Yorke 联合发表了一篇文章，叫做《Period Three Implies Chaos》[2]，《周期三意味着混沌》，让混沌一词映入大众视野，也将混沌这一概念转化为了科学的专业术语。

2.2 初值的敏感性

科学意义上的混沌并非是无序或者混乱，而是没有明显周期与对称，但却具有丰富内层次的有序状态[3]。混沌理论中主要研究的动力系统是那些具有对初始条件敏感的特性的严格确定性动力系统，并且当这些动力系统具有复现的性质时，无法长期预测。初始条件的微小差异，例如由于测量误差或数值计算中的舍入误差，可能会导致此类动力系统产生完全不同的结果，这被称作对初始值的敏感性。为了解释对初始条件的敏感性有多重要，气象学家菲利普·梅里利斯(Philip Merilees)在 1972 年组织了一场会议，洛伦茨在会上展示了他的结果，洛伦茨演讲的标题后来变得十分有名：“可预测性.巴西的蝴蝶翅膀在德克萨斯掀起了龙卷风？”这个标题在许多文章中被引用和修改，变为了目前广为流传的蝴蝶效应[4]。尽管这些系统是确定性的，这种情况仍然可能会发生，这意味着它们的未来行为遵循独特的演变规律[5].并且完全由它们的初始条件决定，不涉及随机元素。换句话说，这些系统的确定性本质并不能使它们可预测[6]。这种行为被称为确定性混沌，或者简称为混沌。在模拟心率、大脑功能等内源性生物节律方面，混沌理论被发现有较大应用，本文所研究的心电散点图就是一个很好的例子。

洛伦兹发现了非线性系统的混沌行为，也就是天气的混沌行为。洛伦兹还用图形描述了他的发现，图 1 展示的图形的是他的第二个发现.奇异 Lorenz 吸引子。

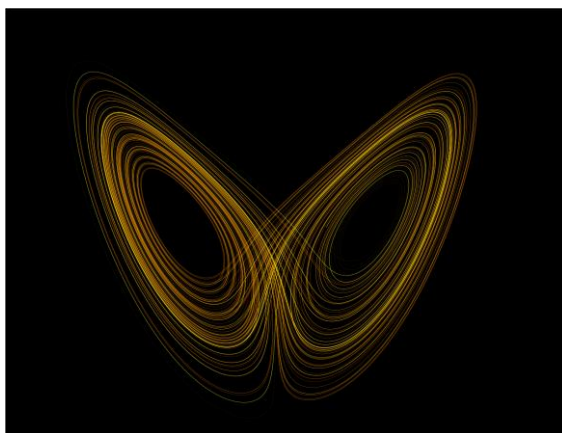


图 1.Lorenz 奇异吸引子

2.3 吸引子

在一个动力系统中，无论是牛顿动力系统或者混沌动力系统亦或是其他动力系统，都存在有吸引子的概念。动力系统吸引子是相空间中具有时间演化轨道特性的区域，可以看作是状态空间的一个子集，受到初始值影响的轨道随着时间不断演化最后会不断地趋近这一个子集。

用于描述动力系统的数学模型通常是一个常微分或者偏微分方程，其自变量多是时间，描述物体随着时间的演化轨迹。在一个多维度的耗散动力系统中，随着时间的流逝，有些变量的影响会逐渐消失耗散，只留下一些稳定的变量持续的作用于物体上，导致产生不同维数的吸引子。一维不动点，二维曲线(极限环)，三维流形，以及导向混沌的奇异分形吸引子。

在常规的耗散动力系统中，吸引子通常有三种，以单摆作为例子，在考虑空气阻力的情况下，单摆会停留在某个点，这里的吸引子是一个不动点，如果不考虑空气阻力的情况下，单摆会做周期运动，吸引子

是一个极限环。在一个复摆所组成的系统中，两个单摆相互制约，相互影响，由于初始角度的微小不同，三个复摆的轨迹有巨大的差异。在这种确定系统中存在的随机运动被称作混沌运动。

混沌运动在几何上的描述被称作分形，或是奇异吸引子，那什么是分形呢。

2.4 分形

曼德勃罗(Mandelbrot)是首位发现并命名混沌中的这种独特几何结构的人，在其所写的《大自然的形几何学》[7]中，谈到“欧几里得几何总是”枯燥乏味”的，原因在于他们无力描绘云彩，山峰，数目或者海岸线的形状。其中云彩不是球体，山峰不是锥体，海岸线不是圆周...”“那他们是什么形状呢，曼德勃罗给他们取名为分形(fractal)，意欲为破碎的，用以更好的描述这个自然界。

为了更好的定义分形，我们需要引入 1918 年德国数学家豪斯多夫(Hausdorff-Besicovitch)所给出的维度定义，此维度定义可以用于度量任意内积空间的维度。

下面利用较为通俗的语言进行描述：

由下图可以看出在一维线段中，将线段平均分割，可以得到与之相似比为 2 的小线段，经过两个这样的小线段可以拼会原来的线段。一个正方形中可以找到与之相似的小正方形，相似比为 2，需要四个小正方形还原大正方形。立方体中找到相似的小正方体，同时需要 8 个进行还原。简单的说，可以利用下式进行计算豪斯多夫维度。

$$a^n = b,$$

$$n = \log_a b,$$

其中 a 是相似比,b 是还原图形所需要的个数。

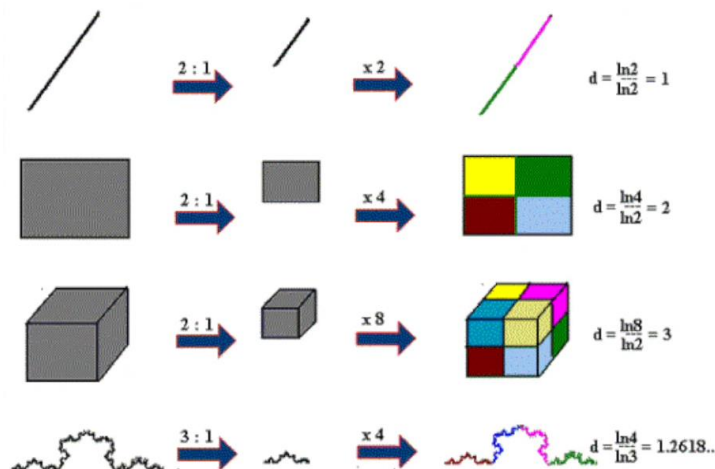


图 2.豪斯多夫维度图解

那什么是分形图呢，曼德勃罗说到，只要某个集合的豪斯多夫(Hausdorff-Besicovitch)维度大于其拓扑维数，那么这样的集合被称作分形。

拓扑维度可以定义为几何体在欧式空间中确定自身位置所需要的数字的个数。其中欧式几何中所有的图形都是都拥有一样的豪斯多夫维度与拓扑维数。

若拓扑维数记为 D ，豪斯多夫维度记为 D_T ， D 总是一个整数， D_T 可以不是一个整数,且满足等式

$$D_T \geq D.$$

分形图拥有很多独特的性质，是传统的欧式几何图形所没有的。例如：

1. 在任意小的尺度上对分形进行观察，都具有分形结构。
2. 自相似性.在较小尺度进行观察的图形与较大尺度进行观察的图形是相似的。
3. 分形维度 $\geq$ 拓扑维度。

可以看出分形图可以较好的描述混沌运动，是一门十分合适的几何语言。在经典动力系统中已知被发现的三类吸引子有. 1. 不动点 2. 极限环 3. 整数维环面。

这三种吸引子是普通的吸引子，一旦被求解出来，这个系统的归宿也就被确定了，并且对于初始值不敏感。在初始值差不多的情况下，才能够对于未来进行预测，这种预测才是可信的。也证明了决定论的可行性。

对于 Lorenz 发现的蝴蝶状混沌吸引子来说，则是完全的不同，在一个参数确定，初始值确定的方程中，最后的运行轨迹完全不是可预测的，只能知道一定在此方程决定的蝴蝶吸引子中。这或许就是 Lorenz 论文《确定性的非周期流》[8]的含义。

2.5 心电信号

本小节将通过数值实验来证明 RR 间期时间序列为混沌数据，其中使用数据为麻省理工大学-以色列贝瑟医院联合公开的数据集。MIT-BIH 数据集。主要分析室上性心律失常、室速性心律失常与房颤数据集。

对于一个由确定性的动力系统产生的时间序列来说，如果出现一个正的 Lyapunov 指数就足以证明此系统为混沌系统。Lyapunov 指数可以用于衡量一个动力系统对于初始值的敏感性。例如，考虑在一个吸引子中两个有着临近初始条件的轨迹，如果此吸引子是混沌的，则轨迹会以指数级别的最大 Lyapunov 指数速率进行收敛或者发散。这个概念也被推广到李雅普诺夫指数谱 $\lambda_i (i = 1, 2, \dots, n)$ 。

为了描述这一类系统,可以考虑初始条件是一个小的 n 维球体，其中 n 是方程的数量(或者，也可以称作状态变量的数量)。随着时间 t 的流动，球体演变成一个椭球体，其主轴以 Lyapunov 指数给出的速率膨胀(或收缩)。如果有正指数的存在，则可以判断为混沌，同时表征了某个特定方向的局部不稳定性。如果动力系统中存在有吸引子，那么整体系统必然是耗散的，即全局稳定的，这样总收缩率必须超过总膨胀率。因此，即使系统中有几个正的李雅普诺夫指数，整个频谱的李雅普诺夫指数总和也是负的，用于保证系统的稳定[9]。

根据文献[A practical method for calculating largest Lyapunov exponents from small data sets][9]，可以逐步的计算此数据集的最大 Lyapunov 指数。文中提出了一种从实践中生成的时间序列中计算 Lyapunov 指数简单易用的方法。此方法对于数据量小的数据集而言，计算是较为快速且可信赖的。

对于某个动力系统而言，如果描述此系统的微分方程是存在的，那么我们可以直接计算出此系统对应的 Lyapunov 谱。首先可以在给定初始条件下求解此微分方程的近似精确解，解向量可以利用 Gauss-Schmidt 正交化将向量重整到正交空间中。可以保证在最快速膨胀的方向只有一个向量在此方向有元素。

然而对于从实验中获得的数据来说，这个方法是无法使用的，因为我们并不能准确知道在心脏动力系统背后隐含的微分方程组是什么样的。所以本文需要的方法为相空间吸引子重构[9]。

定义最大 Lyapunov 指数为

$$d(t) = Ce^{\lambda_1 t} \quad (1)$$

其中 $d(t)$ 是在时间 t 上的平均发散程度， C 是标准化初始的一个常数。

2.5.1 相空间重建

对于某一个可被观测到的一维时间序列而言,重建其隐含的动力系统的相空间是十分必要的。重建好的目标轨迹 X 可以被表示为一个矩阵,其中每一行都是一个相空间的向量。

$$X = (X_1, X_2, X_3, \dots, X_M)^T,$$

其中 X_i 是系统在离散时间 i 的一个状态表示。对于一个具有 N 个点的时间序列 $x_1, x_2, \dots, x_N$ 而言, X_i 由下式给出

$$X_i = (x_i, x_{i+j} \dots x_{i+(m-1)\tau}),$$

其中 J 是相空间重组的时延, m 被称作嵌入维数。因此, X 是一个 $M \times m$ 维的矩阵

$$M = N - (m - 1)\tau.$$

根据 **Takens theorem**, 动力系统中任一分量的演化过程都是由与之相互作用的其他分量决定的。所以,重构系统的相空间只需要考察一个分量,通过固定的延迟点上的观测值重构原系统,重构的空间和原动力系统拓扑等价[10]。由此可以获得 m 的估计,也就是就是 $m > 2n$, 其中 n 是原来空间的维度。

然而对于时间延长 J 的估计通常是一个难以唯一确定的问题。本文将使用相关函数法进行选择。

对于离散混沌时间序列 $x_1, x_2, \dots, x_N$ 而言, 间隔为 $j\tau$ 的自相关函数如下:

$$R_{xx}(j\tau) = \frac{1}{N} \sum_{t=0}^{N-1} x(t)x(t+j\tau),$$

在 j 固定的情况下, 绘制出相关函数与时间间隔 τ 的图像, 找到下降到其初始值的 $1 - 1/e$ 对应的 τ , 就可以获得一个较为准确的估计。

2.5.2 寻找最近邻

在获得一个时间序列的重组相空间之后, 在公式(2)的定义之下, 可以通过搜索与特定参考点 X_k 的距离最小点来找到最近邻点 $X_{\hat{k}}$ 。

$$d_k(i) = \min_{X_{\hat{k}}} \|X_k - X_{\hat{k}}\| \quad (2)$$

其中 $d_k(i)$ 是第 k 个点经过 i 离散时间到最近邻的距离。 $\|\cdot\|$ 为欧几里得距离。 k 与 $\hat{k}$ 需要满足一个额外的条件来保证结果的准确性。最近邻居之间的距离需要大于时间序列的平均周期 p 。平均周期可以被估计为时间序列平均频率的倒数。

$$|k - \hat{k}| > p,$$

这时我们可以将每对邻居视为不同轨迹的临近初始条件。最大的李雅普诺夫指数可以被估计为最近邻居的平均分离率。

$$\lambda_1(i) = \frac{1}{i\Delta t} \frac{1}{(M-i)} \sum_{k=1}^{M-i} \ln \frac{d_k(i)}{d_k(0)},$$

其中 Δt 是时间序列的取样周期。 $d_k(i)$ 是第 k 对最近邻在 i 个离散时间步之后的最短距离。

根据方程(1)所定义的最大 Lyapunov 指数, 我们假设第 k 对最近邻近似地以最大 Lyapunov 指数给出的速率发散。

$$d_k(i) \approx C_k e^{\lambda_1(i\Delta t)} \quad (3)$$

C_k 是初始距离。将方程(3)两边都取对数, 可得

$$\ln d_k(i) \approx \ln C_k + \lambda_1(i\Delta t) \quad (4)$$

方程(4)表示由一组几乎平行的直线 $k = 1, 2 \dots M$, 每一条直线的斜率都近乎正比于 λ_1 , 忽略方程中的常数, $y(i) = \lambda_1 i$, 可得:

$$y(i) = \frac{1}{4t} \langle \ln d_k(i) \rangle,$$

利用最小二乘法对 λ_1 进行估计可以得到最大 Lyapunov 指数。其中 $\langle \ln d_k(i) \rangle$ 表示为对应所有 j 的一个均值。

2.5.3 心电数据应用

目前我所分析数据集为 MIT-BIH 开源数据库中室上性心率失常、室速性心率失常、房颤的数据集。

利用上述方法可取三种数据集中嵌入维度 m 、时延 J 为下表所示。

选取的数值	室上性心率失常	室速性心率失常	房颤
m	1	2	3
J	4	10	10

表 1. 三种疾病选取出的 m 与 j 对应表格

以房颤数据集 $M = 1000$ 的一部分数据

$$X = (X_1, X_2, X_3 \dots X_{1000})^T,$$

作为实例进行阐述具体实现过程。步骤如下:

1. 首先根据上表选取合适的嵌入维度数 3 与时间延长参数 20, 用于进行相空间重构, 可以得到 $1000 - (3 - 1) * 10$ 行 3 列的一个相空间矩阵 X 。
2. 接下来构建距离矩阵 $dmat$ 用于储存每个相空间轨迹之间的间距。其中 $dmat$ 维度为 $1000 * 1000$, 第 i 行第 j 列元素为 $k, \hat{k}$ 两个最近相空间轨迹之间的欧几里得距离。将 $dmat$ 中对角线填充为无穷避免后续计算出错。
3. 为了计算 $d_k(i)$, 在第 k 个相空间轨迹与其相邻的 $\hat{k}$ 邻居满足在时间 i 离散步之后的基础上, 还满足了第 k 个相空间轨迹与第 $\hat{k}$ 之间的差距必须大于平均周期 $0.1M$, 可以利用 $dmat$ 计算出一个关于 i 的一个向量, 将其取对数在对 j 取均值之后得到一个分离值 $\ln(\text{divergence})$ 。
4. 利用不同的时间离散步与分离值进行线性拟合可以得到如下的曲线。

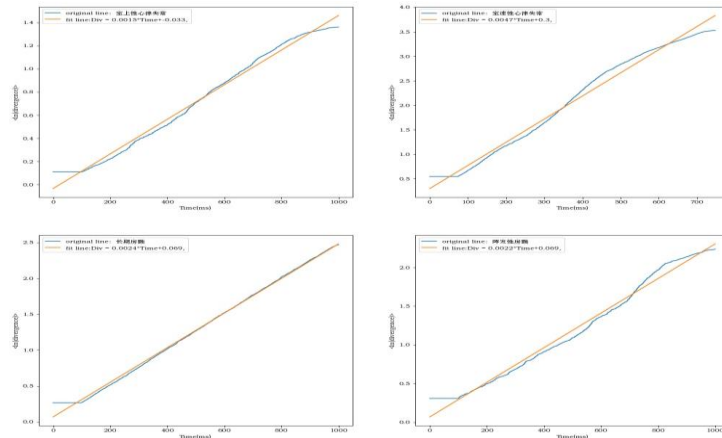


图 3. 室上性、室速性心力衰竭、阵发性、长期房颤混沌性质验证结果图

其中线性曲线的斜率就是该混沌序列的 Lyapunov 指数，可以看出上图中四种疾病的斜率均为大于 0 的正数

疾病	室上性心率失常	室速性心率失常	长期房颤	阵发性房颤
图像斜率	0.015	0.0047	0.0024	0.0022

表 2. 室上性、室速性心力衰竭、阵发性、长期房颤混沌性质验证的斜率表格

利用 MIT-BIH 三种病例的数据我计算 Lyapunov 指数，可以看出，所拥有的四种疾病 Lyapunov 指数拥有一个正数，可以肯定其为混沌序列。

第三章 心电散点图

3.1、何为心电散点图

在人类的心脏系统中，心脏的跳动是由非常多的机制耦合在一起相互作用的结果，并不能完全由某个定理唯一的确定，也不是完全随机的跳动。具有代表性的心电信号是一种非平稳的生理信号，代表心脏的电活动。它不仅可以用于查找心跳中的病理模式，还可以用于测量心跳的规律性以及诸如精神压力之类的其他状况。很多实验表明，心脏的生理节律既不同于周期震荡，也非完全随机，而是具有混沌特性，在确定系统中表现为貌似随机的行为[11]。以长时间的体表心电信号为代表的人体生理节律在不同尺度下表现为。分形结构的自相似性、相空间图动态特性接近奇怪吸引子[12]。不同的心脏疾病展示的性质特点不完全相同。通过对心电信号进行分析可以对患者的疾病类型进行诊断。许多心脏异常将在心电图上表现出来，包括心律失常，其中有心动过速，心动过缓，室上性心律失常和心室性心律失常。心律失常诊断的基础是识别正常与异常的个体心跳，并根据心电图形态将其正确分类为不同的诊断。心跳可细分为五类，即非异位、室上异位、心室异位、融合和未知心跳。在 ECG 上区分这些心跳既具有挑战性又耗时，因为这些信号通常被噪声破坏[13]。

由于心电信号具有复杂和高混沌的特点，即使对于专家来说，解释心电信号也既耗时又费力。因此，需要计算机辅助方法来减轻人的负担，从而减少内部差异引起的错误。近年来，深度学习在心电图分类研究中表现出色。其分层架构可实现更高层次的特征，其强大的特征提取能力有助于分类项目。最新研究可以达到比专家手动分类更高的准确性和效率。心率变异性[14]分析是目前临床认可的用于定量评价自主神经功能的主要方法。作为一种无创性检测手段，分析对于心脏疾病及其他自主神经相关疾病的诊断、生理状态的评估都具有重要的临床价值。常用分析方法包括时域、频域和非线性方法。其中，非线性分析方法被认为更有助于揭示心脏动力系统的本质特征。但是，目前大多数非线性方法还不够成熟，未被临床应用所接受。

对于心脏信号的研究主要基于标准的心电图(electrocardiogram, ECG)，即心电的电压幅度随时间变化的信号，医学界将从心电信号中提取的心搏信号称为心率变异信号(Heart rate variability,HRV)[15]，对心电信号进行特征提取并进行分析识别的主要手段有两种[14]。

第一种基于其波形结构进行时域或者频域的线性分析，

(1)时域分析:抽取 ECG 信号中 P 波，QRS 复合波，RR 间期等来对特定的疾病进行直接判断，例如上述波形的频率，波数，持续时间都是心房颤动(AF)、早搏房收缩需要考虑的特征。RR 间期是其中最重要的一个特征。

(2)频域分析:利用快速傅里叶变化对 ECG 信号进行数学变换，其中主频峰值、主峰以下的光谱含量、峰值波段中的光谱浓度等特征已显示在心律失常检测中的有效性。其中可以采用不同的方法对于获取的 qrs 波形进行去噪，其中噪音可能由于各种原因被引入，例如检测心电信号的机器，和病人连接的电机处都是很有可能导致噪音被引入。对不同形式的噪音，有不同的去噪方法[16]。

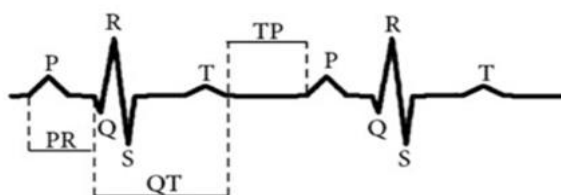


图 4.ECG 信号对应的波形名称

第二种则是非线性的特征提取，由于线性特征无法展现心电信号的动态特征，需要非线性特征来展现 ECG 信号的混沌特性以及心率变异性(Heart rate variability,HRV)。主要方法有：

(1)熵.利用信息熵，排列熵对疾病进行分类。

(2)洛伦兹散点图(庞加莱散点图)是利用 $RR(i), RR(i+1)$ 对 RR 间期数据进行绘制的散点图，利用该图像特征可以对疾病进行分类。

(3)隐马尔科夫模型.隐马尔科夫模型为基于观测预测状态机器状态的统计模型。RR 序列可被建模为三态 Markov 过程进行分析。

通过对心电信号进行分析可以对患者的疾病类型进行诊断。RR 间期是指人类心跳两个 R 波之间的时间差，如图 5.1 所示。其数值可以表示人类心跳的一次心动周期时间。如图 5.2 所示，将动态心电图 ECG 中的 RR 间期数据间隔一位化为图像坐标，在坐标系中迭代作图制成最后的心电图像。通过绘制心电散点图能够将充分利用心电数据，将一般心电图难以识别的疾病心电信号变得更加的突出。



图 5.1 RR 间期

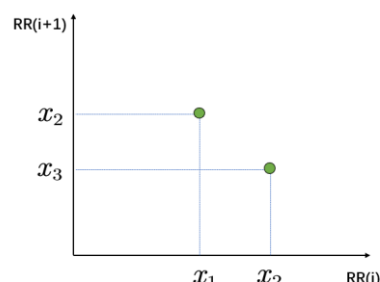


图 5.2 绘制二维心电散点图示意图

3.2、本文提出的改进方法

本文对于方法的改进基于文献[3]中提出的二维心电散点图，通过将散点的频次进行更好的估计使得产生的三维心电散点图更加的具有特征以及容易识别。同时此方法已申请专利并已获授权[17]。

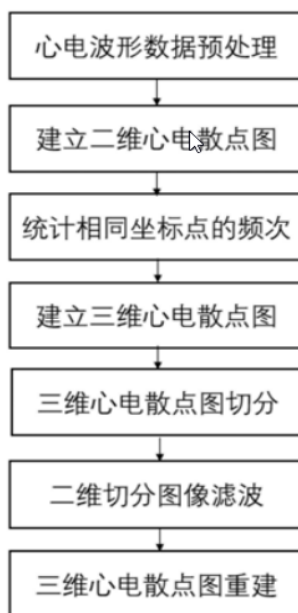


图 6. 三维散点图像作图方法

步骤 1. 心电波形图数据预处理

从美国麻省理工学院提供的 MIT-BIH 心电数据库中提取心电数据，结合所收集到的数据。在心电波形图原始数据中，提取其中的 RR 间期数据，并储存在一维数组：

$$[x_1, x_2, \dots, x_i, \dots, x_n],$$

本例选取 20000 个 RR 间期数据作为一组绘制图像。其中， x_i 表示第 i 个 RR 间期数据， n 表示 RR 间期数据个数， $0 < i \leq 20000$ 。

步骤 2. 建立二维心电散点图

建立维度为 $m * m$ 的二维心电散点图平面坐标系。横轴为 X ，纵轴为 Y 。其中 $m = 200$ 。其上的散点为

$$P_1(x_1, x_2), P_2(x_2, x_3), \dots, P_j(x_j, x_{j+1}), \dots, P_{n-1}(x_{19999}, x_{20000}),$$

其中 $P_j(x_j, x_{j+1})$ 表示第 j 个二维散点， x_j 表示 $P_j(x_j, x_{j+1})$ 的 X 坐标， x_{j+1} 表示 $P_j(x_j, x_{j+1})$ 的 Y 坐标， $0 < j \leq 19999$ ；如图 7 所示为二维心电散点图实例。

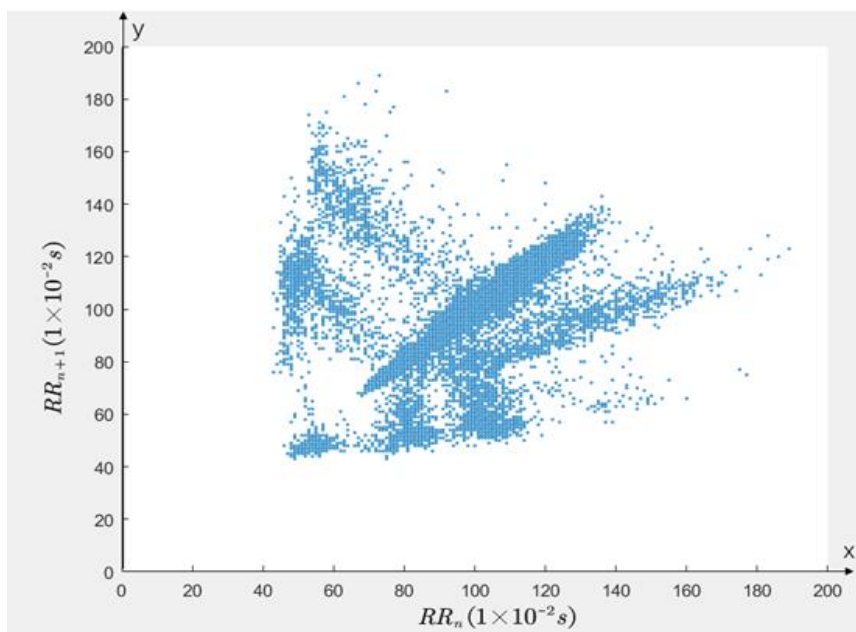


图 7.二维心电散点图

步骤 3. 统计相同坐标散点的频次

对所述二维心电散点图中相同坐标散点 $P(x_j, x_{j+1})$ 的出现频次进行统计，并记录为 $f(P(x_j, x_{j+1}))$ 。其中， $f(P(x_j, x_{j+1}))$ 表示所述相同坐标散点 $P(x_j, x_{j+1})$ 的出现频次。

步骤 4. 建立三维心电散点图

所述心电散点图可用如下点阵表示。使用 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))$ 表示其中的某一个三维散点。则 x_j 表示 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))$ 的X坐标， x_{j+1} 表示 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))$ 的Y坐标， $f(P(x_j, x_{j+1}))$ 表示 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))$ 的Z坐标， $0 < j \leq 19999$ 。

如图 8 所示为三维心电散点图实例。

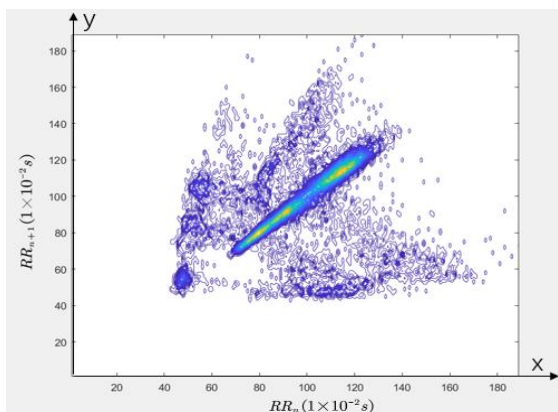


图 8.三维心电散点图

步骤 5. 三维散点图切分

由于所述的三维散点图难以进行滤波处理，于是考虑将三维散点图按照 X 坐标轴沿着 YOZ 所成平面切分，由于坐标轴大小为 $200 * 200$ ，则共可以将三维图像分割为200份二维图像，每一份图像保留第三维的频次信息。

步骤 6. 二维散点图图像滤波

对所述三维心电散点图进行图像滤波处理，所采用的滤波条件为：

$$Y = \min \left(\sum \left((u_p - g_p)^2 + \lambda \left(a_x, p(g) \left(\frac{\partial u}{\partial x} \right)^2 \right) + \lambda \left(a_y, p(g) \left(\frac{\partial u}{\partial y} \right)^2 \right) \right) \right),$$

其中， u 代表输入的图像， g 代表输出的图像，下标 p 代表像素点的空间位置。 Y 表示优化函数， $\min$ 表示取最小值， Y 需要取最小值，代表着输入图像与输出图像的信息值尽可能的小。 a_x ， $p(g)$ 和 a_y ， $p(g)$ 代表平滑项的权重，利用了拉格朗日数乘法， λ 是拉格朗日乘数，对图像坐标点的 x, y 坐标分别求偏导是拉格朗日函数建立的前提。

其中参数 $a_x, p(g)$ 取值为：

$$a_x, p(g) = \left(\left| \frac{\partial l}{\partial x}(p) \right|^2 \right)^{-1},$$

参数 a_y ， $p(g)$ 取值为：

$$a_y, p(g) = \left(\left| \frac{\partial l}{\partial y}(p) \right|^2 \right)^{-1}.$$

步骤 7. 三维散点图重建

使用原点阵的排列方式将步骤 6 中经过滤波的 200 张二维图像进行排列连接，重新组成点阵如下。

$$\begin{bmatrix} NP(1,1,f(P(1,1)))^* & NP(1,2,f(P(1,2)))^* & \dots & NP(1,200,f(P(1,200)))^* \\ NP(2,1,f(P(2,1)))^* & NP(2,2,f(P(2,2)))^* & \dots & NP(2,200,f(P(2,200)))^* \\ \vdots & \vdots & \vdots & \vdots \\ NP(200,1,f(P(200,1)))^* & NP(200,2,f(P(200,2)))^* & \dots & NP(200,200,f(P(200,200)))^* \end{bmatrix}$$

使用 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))^*$ 表示滤波后的某一个三维散点。则 x_j 表 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))^*$ 的 X 坐标， x_{j+1} 表示 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))^*$ 的 Y 坐标， $f(P(x_j, x_{j+1}))$ 表示 $NP(x_j, x_{j+1}, f(P(x_j, x_{j+1})))^*$ 的 Z 坐标， $0 < j \leq 19999$ 。

3.3、训练神经网络验证三维心电散点图优势

近些年来，基于联结主义而提出的深度神经网络在人工智能领域十分火热。尤其在图像识别领域有着较高的识别率以及落地价值。针对专利[17]中提出的新方法，本文将使用 vgg16 模型[18]对二维以及三维心电散点图进行识别。简要的判断各种疾病之间识别的准确率。

数据准备:从开源数据库 MIT-BIH 网站下载数据,获取疾病室上性心率失常,室速性心率失常,房颤的原始数据集。所获数据基本为医院通过多导联传输记录的电信号。利用该网站开源工具包 `wfdb-python` 对于原始信号进行处理。利用 XQRS 算法可以将其中的 QRS 波群探测出来,然后对其进行一阶差分运算,将其中的 RR 间期数据提取出来,储存为 python 中 `pkl` 文件备用。

数据处理:读取各种疾病数据的 `pkl` 数据并选取3000个点一张图进行绘制二维图像以及三维图像。下表为三种数据可绘制而成的图像数量。

疾病	室上性心率失常	室速性心率失常	房颤
图像数量	118	70	2722

表 3. 三种疾病绘制而成的二维或三维图像数量

由于初始图像数据集数量具有显著差异,在具体网络训练中将数据量较少的图像做了一定的数据增广,具体为使用`pytorch`中`tencrop`函数,获取1张图像的中间,左上,右上,左下,右下,以及翻转之后的图像,可以扩充到10张。用于尽量减少数据数据的长尾效应。

将所有图片数据随机打乱后取前70%作为训练集,后20%作为验证集训练`vgg`神经网络,再利用最后10%作为测试集得到最后的实验精度。

损失函数使用交叉熵函数

$$H(p, q) = - \sum p(x) \log q(x),$$

优化器使用随机梯度下降,学习率为0.01,动量为0.9。批量大小 `BatchSize` 选择 32,最后训练两个 epoch。

训练结果:二维图像与三维图像在划分测试集的实验准确率如下

使用的网络	室上性心率失常	室速性心率失常	房颤
VGG 二维图像	79.6%	83.4%	86%
VGG 三维图像	83%	88.9%	92.5%

表 4. 三种疾病在相同分类器 VGG 下在测试集的准确率

第四章 总结与展望

本文从心脏跳动的混沌本质介绍开始，对混沌理论的产生背景，独有性质如分形，吸引子等概念进行了阐述。为了研究心脏疾病并通过本文作者提出的专利[17]对不同的心脏疾病进行分类。首先通过第一个实验经历相空间重构等一系列操作证明了心脏数据确实为混沌序列，因为其最大 Lyapunov 指数为正数。其次利用疾病数据获取 RR 间期序列，绘制二维三维心电散点图。将问题转化为对图像进行分类。在通过 pytorch 工具构建 vgg16 深度学习模型对所拥有的三类疾病数据进行分类。可以看出在当前数据集划分上对于专利[17]中提出的算法相比于文献[3]具有更高的准确率。

未来期望可以与医院进行合作，扩大我们的数据集。收集建立多种心脏病的 RR 间期数据集，将在此三种疾病诊断中所取得的成果做出进一步的扩展，应用于更多种心脏疾病的诊断当中。

参考文献

- [1] POINCARÉ H. Les méthodes nouvelles de la mécanique céleste: 卷 3[M]. Gauthier-Villars, 1899:1-334
- [2] LI T Y. YORKE J A. Period three implies chaos[M] The theory of chaotic attractors. Springer, 2004: 77-84.
- [3] 李方洁. 《心电散点图》: 卷 28[M]. 中国心脏起搏与心电生理杂志, 2014: 4-5.
- [4] KELLERT S H. In the Wake of Chaos: Unpredictable Order in Dynamical Systems[M]. University of Chicago Press, 1994:1-169.
<https://doi.org/10.7208/9780226429823>.
- [5] ZALTA E N. NODELMAN U. ALLEN C. Stanford encyclopedia of philosophy[M]. Metaphysics Research Lab, Center for the Study of Language; Information 1995:1-16.
- [6] WERNDL C. What are the new implications of chaos for unpredictability?[J]. The British Journal for the Philosophy of Science, 2020:195-220.
<https://doi.org/10.1093/bjps/axn053>.
- [7] MANDELBROT B B. MANDELBROT B B. The fractal geometry of nature: 卷 1[M]. WH freeman New York, 1982:1-16.
- [8] LORENZ E N. Deterministic nonperiodic flow[J]. Journal of Atmospheric Sciences, 1963, 20(2): 130-141.
- [9] ROSENSTEIN M T. COLLINS J J. DE LUCA C J. A practical method for calculating largest Lyapunov exponents from small data sets[J]. Physica D: Nonlinear Phenomena, 1993, 65(1-2): 117-134.
- [10] TAKENS F. Detecting strange attractors in turbulence[M] Dynamical systems and turbulence, Warwick 1980. Springer, 1981: 366-381.
- [11] 霍铨宇. 基于庞加莱散点图和递归图的心率变异性研究[D] 南京大学. 2014: 136-142.
- [12] LIU X. WANG H. LI Z. et al. Deep learning in ECG diagnosis: A review[J]. Knowledge-Based Systems, 2021, 200-227: 107187.
- [13] RAJPURKAR P. HANNUN A Y. HAGHPANAHI M. et al. Cardiologist-level arrhythmia detection with convolutional neural networks[J]. arXiv Preprint arXiv:1707.01836, 2017:1-15.
- [14] LE T H N. LE T M. LE T Q. et al. Feature extraction techniques for automatic detection of some specific cardiovascular diseases using ECG: a review and evaluation study[C] International Conference on the Development of Biomedical Engineering in Vietnam. Springer, 2018: 543-549.

- [15] ALTAN G. KUTLU Y. ALLAHVERDI N. A multistage deep belief networks application on arrhythmia classification[J]. International Journal of Intelligent Systems and Applications in Engineering, 2016: 222-228.
- [16] MAKHZANI A. FREY B. K-sparse autoencoders[J]. arXiv Preprint arXiv:1312.5663, 2013:1-9.
- [17] 汪中旭;杨德乾;付明磊. 一种心电散点图三维图像增强方法[P]. 中国专利, 2019.1-11
- [18] SIMONYAN K. ZISSERMAN A. Very deep convolutional networks for large-scale image recognition[J]. arXiv Preprint arXiv:1409.1556, 2014:1-14.

致 谢

时光荏苒岁月如梭，大学四年弹指间已然度过。此份毕业论文的完成离不开老师，同学，朋友的真诚帮助。

首先需要感谢的是谢聪聪老师对我论文内容以及格式的专业指导，以及研究生导师张立军老师的专业指导。从而使我顺利完成本次论文。

在这四年期间，本人勤奋好学，在不断的自我鞭策激励下前进，感谢我自己不断地努力以及始终如一的前进。

感谢浙江工业大学理学院老师们在这四年中教会我的专业知识技能以及敦敦教诲。在我最迷茫的时候，是他们给我指出了一条条明路，让我有了更加清晰的人生认识，这会是我一生珍藏的宝藏。

感谢我的大学同学以及室友，他们是我前进路上相互鼓励同伴与竞争的对手，没有他们的相互督促，我不会取得这么大的成绩。

最后感谢我的家人，他们在任何时候都会坚定的支持我，是我前进路途的避风港。

附 录

```

import os
import numpy as np
import numpy as np
import matplotlib.pyplot as plt

datapath = 'E:\大三项目数据\RR 间期数据'
data, type = {}, {}
for root, dirs, files in os.walk(datapath):
    if files: # 判断列表是否为空，此为有元素
        for ele in files:
            p = root + "/" + ele
            data[ele[:-4]] = np.array([int(i) for i in np.loadtxt(p)])
            type[root[17:]] = [i[:-4] for i in files] # 把数据的类型进行储存
d = np.loadtxt(p)

def get_phase_space_matrix(data, m, J):
    """
    input:
        data: 需要处理的时间序列 --> np.ndarray
        m: indicates the embedding dimension --> int
        J: indicates the lag --> int
    output:
        生成数据的 matrix
    """
    N = len(data) # N 数据长度
    M = N - (m-1)*J
    matrix = np.zeros(m)
    index = []

    for i in range(M):
        ind = list(range(i, i+(m-1)*J+1, J)) # start stop step
        index.append(ind)
        matrix = np.vstack([matrix, data[ind]]) # 竖着进行堆叠
    dmat = np.zeros(len(matrix)) # 必须放在这里，matrix 生成之后才做另一个
    # 计算最近邻距离
    for i in range(len(matrix)):
        temp_dist = [np.sqrt(sum((matrix[i]-matrix[j])**2))
                     for j in range(len(matrix))]
        dmat = np.vstack([dmat, temp_dist])

    return index, matrix[1:], dmat[1:] # 把一开始的 0 去除

```

```

def get_average_dist(dmat, i_interval):
    mean_period = int(0.1*dmat.shape[0])
    aver_dist = np.array([])
    for i in range(len(dmat)): # i_interval indicates 间距/半径
        # aver_dist = np.append(aver_dist,
        # min(dmat[i,max(0,i-i_interval):i+i_interval+1])) # 以 i_interval 为半径去找一个最小
    值
        aver_dist = np.append(aver_dist, min(dmat[i, min(dmat.shape[0]-1, (i+mean_period if
    (i+mean_period) # 保证至少是有一定的间隔的
        > (i+i_interval) else i+i_interval)):])) # 以 i_interval 之
    后的计算最小值
        # 把数据中的 0 换成 1, 这样可以在 log 计算时消去
        aver_dist = [1 if i == 0 else i for i in aver_dist]
        return np.mean(np.log(aver_dist))

need_verify_data = ["826", 'cu01', '00', '04015']
len_need_verify_data = [len(data[i]) for i in need_verify_data]
m, J = [1], [10] # range(1,10,5), range(1,20)
# 用于绘制子图的数据累积
cum_X, cum_Y, cum_z, cum_ab = [],[],[],[]
for k_i in range(4):
    k = need_verify_data[k_i]
    for m_i, J_i in zip(m, J):
        ind, mat, dmat = get_phase_space_matrix(
            data[k][(len(data[k])/2) if (len(data[k])/2) > 1000 else 0:(len(data[k])/2+1000)],
m_i, J_i)
        for i in range(len(dmat[1:])): # 把对角线变为无穷大
            dmat[i, i] = np.inf
        y = np.array([])
        for i in range(len(dmat)):
            y = np.append(y, get_average_dist(dmat, i))
        # 将下面的曲线进行线性拟合
        x = list(range(0, len(dmat)))
        l = len(y)
        fstring = f'm = {m_i}, J = {J_i}'
        a, b = np.polyfit(x, y, 1)
        z = a*np.ones(l)*x+b*np.ones(l)
        plt.plot(x, y, label="original line")
        plt.plot(x, z, label=f'fit line:Div = {a:.2}*Time+{b:.2},')
        plt.xlabel("Time(ms)")
        plt.ylabel("<ln(divergence)>") # f'dis = {a:.5}*i+{b:.5}',xy=(600,3.5)
        plt.legend()

```

```

plt.show()
cum_X.append(x)
cum_Y.append(y)
cum_z.append(z)
cum_ab.append((a, b))

import torchvision.models as models
import torch.optim as optim
import torch.nn as nn
from torchvision import transforms
from matplotlib import pyplot as plt
from PIL import Image
import torch
import os
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import Counter
import torchvision # .transforms as transforms
from torch.utils.data import Dataset, DataLoader
os.environ["CUDA_VISIBLE_DEVICES"] = '0,1,2,3,4,5,6,7'
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
datapath = r'/pub/data/yangpf/data/RR 间期数据'
datapath = r'E:\大三项目数据\RR 间期数据'
data, type, dir = {}, {}, []

class SetFileFolder: # 此类用于创建文件夹
    def __init__(self, path, filename):
        self.path = path
        self.filename = filename
        for fn in filename: # 创建文件夹
            # file_name = str(i)
            dir_name = self.path + '/' + str(fn)
            if fn not in os.listdir(self.path): # 文件夹名称不存在才创建
                os.mkdir(dir_name)

for root, dirs, files in os.walk(datapath):
    if files: # 判断列表是否为空，此为有元素
        # for ele in files:
        #     p = root + "/" + ele
        #     data[ele[:-4]] = np.array([int(i) for i in np.loadtxt(p)])
        type[root[15:]] = [i[:-4] for i in files] # 把数据的类型进行储存

```

```

for root, dirs, files in os.walk(datapath):
    if dirs: # ['paroxysmal_atrial', 'supraventricular_arrhythmia', 'Ventricular_arrhythmia',
'long_term_pf']
        lb_mp = {ind: disease for ind, disease in zip(range(0, 4), dirs)}
lb_mp # = {0: '1 长期房颤', 1: '2 室上性心律失常', 2: '3 室速性心律失常', 3: '4 阵发性房颤'}

#绘制三维散点图

def Draw3dplot(data, type, name, savep='/pub/data/yangpf/3Dplot', statistic=3000,):
    """
    input:
        name:str 输入的是需要画图的 txt 文件名称
        data: {name:list of RR data}
        type: {type of disease:list of the RR data names}
    """
    lines1 = [str(i) for i in data[name]]
    long = len(lines1)
    savepath = os.path.join(
        savep, [lb for lb, content in type.items() if str(name) in content][0])

    listcor = []
    listcor.append(lines1[:-1])
    listcor.append(lines1[1:])
    # 将矩阵进行了转置变成了坐标的形式
    matrixtr = list(map(list, zip(*listcor)))
    # 所有的坐标的坐标都变成了
    matrixjoin = ['.'.join(i) for i in matrixtr]
    matrixjoin = [int(x) for x in matrixjoin] # 所有的坐标组合
    b = dict(Counter(matrixjoin))
    cornum = {key: value for key, value in b.items() if value >
        1} # 展现重复元素和重复次数
    j = 0
    # 如果遇到总长度小于一个单位的话可以直接画图 # 以 s 为一个单位进行画图
    for i in range(0, max(long, (long // statistic) * statistic)+1, min(long, statistic)):
        xrr = lines1[i:i + statistic - 1] # x 坐标
        yrr = lines1[1 + i:statistic + i] # y 坐标
        # 元组不可迭代只能通过字符串的处理方式来解决,将 xy 坐标变成了
        z = ['.'.join(x) for x in zip(xrr, yrr)]
        xrr = [int(i) for i in xrr]
        yrr = [int(i) for i in yrr]
        zrr = []
        for k in range(len(z)):
            zrr.append(cornum.get(int(z[k]))) # 不能将这个赋值给某个元素 zrr 坐标搞出来了

```



```

Matrix = np.zeros((200, 200))
for i in range(len(zrr)):
    if xrr[i] < 200 and yrr[i] < 200:
        Matrix[xrr[i]][yrr[i]] = zrr[i] # 将零矩阵填充为频次矩阵
x = list(range(1, 201))
y = list(range(1, 201))
plt.contour(x, y, Matrix, 100, linewidths=0.7)
# plt.show()
plt.xticks() # 去除刻度
plt.yticks() # 去除刻度
if j == 0:
    print('画图中')
    print(">", end=" ")
    plt.savefig(savepath + '/' + str(name) + '第%d 张' %
                (j + 1), dpi=300) # + str(name)
    j += 1
plt.close()

for name, d in data.items():
    Draw3dplot(data, type, name, savep=r'E:\大三项目数据\RR3D', statistic=3500)

# 数一数画了多少张图片
thirdD_path = '/home/yangpf/data/3Dplot'
num_dic = {v: 0 for v in lb_mp.values()}
for root, dirs, files in os.walk(thirdD_path):
    if files: # 判断列表是否为空，此为有元素
        for ele in files:
            num_dic[root[25:]] += 1
num_dic, sum(num_dic.values())

# 数据标签的划分

class PicDataset(Dataset):
    """
    1. 将数据集的标签与数据进行储存
    2. 划分训练集、测试集
    """

    def __init__(self, mode, randseed, split_n=0.9, picdir=r'/pub/data/yangpf/3Dplot',
transform=None):
        self.mode = mode
        self.picdir = picdir
        self.randseed = randseed

```

```

self.split_n = split_n
self.transform = transform
self.data_info = []
self.data_info = self._get_img_info() # 用来装一个个的元组，分别是图片路径与类别编号

def _get_img_info(self):
    """
    多写一个函数的目的在于可以提前将 data_info 进行
    """
    for disease in os.listdir(self.picdir):
        p = os.path.join(self.picdir, disease)
        for path in os.listdir(p):
            picpa = os.path.join(p, path)
            self.data_info.append(
                (picpa, [k for k, v in lb_mp.items() if disease == v][0]))
    print(len(self.data_info))
    # 用于分割训练集测试集
    random.seed(self.randseed)
    random.shuffle(self.data_info)
    splitinx = int(len(self.data_info)*self.split_n)
    if self.mode == "train":
        self.data_info = self.data_info[:splitinx]
    elif self.mode == "test":
        self.data_info = self.data_info[splitinx:]
    else:
        raise Exception("Invaild mode {}".format(self.mode))
    return self.data_info

def __getitem__(self, index):
    pathimg, lb = self.data_info[index]
    img = Image.open(pathimg).convert("RGB")

    if self.transform != None:
        img = self.transform(img)
    return (img, lb)

def __len__(self):
    return len(self.data_info)

norm_mean = [0.485, 0.456, 0.406] # imagenet 训练集中统计得来
norm_std = [0.229, 0.224, 0.225]
traintrans = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(norm_mean, norm_std)
])

```

```

))
normalize = transforms.Normalize(norm_mean,norm_std)
validtrans = transforms.Compose([
    transforms.TenCrop(size = 1000),
    # 需要一个标准化
    transforms.Lambda(lambda chops: torch.stack(
        [normalize(transforms.ToTensor()(chop)) for chop in chops] ))
])

# traintensor,trainlb
traindata = PicDataset("train",randseed=0, picdir=
'/home/yangpf/data/3Dplot',transform=traintrans)
testdata = PicDataset("test",randseed=0, picdir= '/home/yangpf/data/3Dplot',transform=
validtrans)

# hyperparameter
Batch_size = 20
num_classes = 4
MAX_EPOCH = 5
log_interval = 2
val_interval = 1

# 构建 dataloader
trainloader = DataLoader(
    dataset=traindata, batch_size=Batch_size, shuffle=True)
validloader = DataLoader(dataset=testdata, batch_size=4)

# 将数据进行一定的处理

def data_process(img_path):
    norm_mean = [0.485, 0.456, 0.406]
    norm_std = [0.229, 0.224, 0.225]
    img_transform = torchvision.transforms.Compose([
        torchvision.transforms.Resize((200, 200)),
        # torchvision.transforms.TenCrop((99,99)),
        torchvision.transforms.ToTensor(),
        torchvision.transforms.Normalize(norm_mean, norm_std)
    ])
    # path to img
    img_rgb = Image.open(img_path).convert('RGB')

    # img to tensor
    img_tensor = img_transform(img_rgb)

```

```
img_tensor.unsqueeze_(0)
img_tensor = img_tensor.to(device)
# print(img_rgb.mode)
return img_tensor, img_rgb

def get_vgg(path_state_dict, device):
    model = models.vgg16()
    dict = torch.load(path_state_dict)
    model.load_state_dict(dict)
    model.eval() # 暂不清楚用于做什么
    model.to(device)
    return model

# os.path.join(BASE_DIR, "Data", "vgg16-397923af.pth")
path_state_dict = r'/home/yangpf/code/VGG/B_VGG/pre_train/vgg16-397923af.pth'
vgg16_model = get_vgg(path_state_dict, device)

num_fts = vgg16_model.classifier._modules["6"].in_features
vgg16_model.classifier._modules["6"] = nn.Linear(
    num_fts, num_classes) # 把最后一层改为了全连接层

# 损失函数
criterion = nn.CrossEntropyLoss()
# 优化方法
optimizer = optim.SGD(vgg16_model.parameters(), lr=0.01, momentum=0.9)
# 损失策略
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=1, gamma=0.1)

train_curve = list()
valid_curve = list()

for epoch in range(0, MAX_EPOCH):
    loss_mean = 0.
    correct = 0.
    total = 0.
    vgg16_model.train()
    for i, data in enumerate(trainloader):

        # forward
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = vgg16_model(inputs)
        print(device)
```

```

# backward
optimizer.zero_grad()
loss = criterion(outputs, labels)
loss.backward()

# update weights
optimizer.step()

# 统计分类情况
_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).squeeze().cpu().sum().numpy()

# 打印训练信息
loss_mean += loss.item()
train_curve.append(loss.item())
if (i+1) % log_interval == 0:
    loss_mean = loss_mean / log_interval
    print("Training:Epoch[{:0>3}/{:0>3}] Iteration[{:0>3}/{:0>3}] Loss: {:.4f} Acc:{:.2%}"
lr:{}".format(
        epoch, MAX_EPOCH, i+1, len(trainloader), loss_mean, correct / total,
scheduler.get_last_lr()))
    loss_mean = 0.

scheduler.step() # 更新学习率

# validate the model
if (epoch+1) % val_interval == 0:

    correct_val = 0.
    total_val = 0.
    loss_val = 0.
    vgg16_model.eval()
    with torch.no_grad():
        for j, data in enumerate(validloader):
            inputs, labels = data
            inputs, labels = inputs.to(device), labels.to(device)

            bs, ncrops, c, h, w = inputs.size()
            outputs = vgg16_model(inputs.view(-1, c, h, w))
            outputs_avg = outputs.view(bs, ncrops, -1).mean(1)

            loss = criterion(outputs_avg, labels)

```

```
_, predicted = torch.max(outputs_avg.data, 1)
total_val += labels.size(0)
correct_val += (predicted == labels).squeeze().cpu().sum().numpy()

loss_val += loss.item()

loss_val_mean = loss_val/len(validloader)
valid_curve.append(loss_val_mean)
print("Valid:\t Epoch[{:0>3}/{:0>3}] Iteration[{:0>3}/{:0>3}] Loss: {:.4f}
Acc:{:.2%}".format(
    epoch, MAX_EPOCH, j+1, len(validloader), loss_val_mean, correct_val / total_val))
vgg16_model.train()

train_x = range(len(train_curve))
train_y = train_curve

train_iters = len(trainloader)
valid_x = np.arange(1, len(valid_curve)+1) * train_iters * \
    val_interval # 由于 valid 中记录的是 epochloss，需要对记录点进行转换到 iterations
valid_y = valid_curve
```